线性基

2026.4.3又遇到线性基了，感觉是个噩梦啊，虽然之前遇到的线性基全都赛时过了，但就是稀里糊涂的，WA的不知道为什么会WA，AC的不知道为什么能AC

2024CCPC网络赛J：找最小 - Problem - QOJ.ac

2024CCPCFINAL热身赛D：Fish only have a three-second memory span / 鱼的记忆只有 3 秒 - Problem - QOJ.ac

2025ICPC上海G：Gemcrate - Problem - Universal Cup Judging System

2025春季hdu3-1005：1005 月球异或

线性基基本概念

定义：给定数集 S ，以异或运算张成的数集与 S 相同的极大线性无关集，称为原数集的一个线性基

通俗地说，线性基是一个数的集合。每个序列都拥有至少一个线性基，取线性基中若干个数异或起来可以得到原序列中的任何一个数

线性基重要性质

性质一：原序列的任意一个数都可以由线性基内部的一些数异或得到

性质二：线性基内容的任意数异或起来都不能得到 0

性质三：线性基内部的数个数唯一；且在保持性质一的前提下，数的个数是最少的

线性基基本操作

插入

```
void insert(int x) {
    for(int i = B - 1; i >= 0; i--)
        if(x >> i & 1) {
            if(!p[i]) {
                p[i] = x;
                break;
            }
            x ^= p[i];
        }
}
```

求最大值

```
int max_xor() {
    int res = 0;
    for(int i = B - 1; i >= 0; i--)
        if((res ^ p[i]) > res)
            res ^= p[i];
    return res;
}
```

求最小值：先判断是否有元素不能插入线性基，如果有，最小值就是 0 ，否则就是最小的 p_i

```
int cnt_xor() {
    int cnt = 0;
    for(int i = 0; i < B; i++)
        if(p[i])
            cnt ++;
    return cnt;
}

int min_xor() {
    if(cnt_xor() < n) return 0;
    for(int i = 0; i < B; i++)
        if(p[i])
            return p[i];
}
```

求第 k 小值：考虑进一步简化线性基，显然一个线性基肯定可以表示为若干个形如 2^i 的数，可以在 $O(\log_2^2 n)$ 的时间内解决。经过这一步操作后，设线性基内共有 cnt 个数，则它们一共可以表示出 2^{cnt} 个数，将 k 作二进制拆分，异或上对应的线性基即可

```
vector<int> p1;
void exchange() {
    p1.clear();
    for(int i = 0; i < B; i++) {
        for(int j = i - 1; j >= 0; j--)
            if(p[i] >> j & 1)
                p[i] ^= p[j];
        if(p[i]) p1.push_back(p[i]);
    }
}

int min_k_xor() {
    exchange();
    int cnt = p1.size();
    if(cnt < n && k == 1) return 0;
    if(cnt < n) k --;
    if((1 << cnt) < k) return -1;
```

```

    int res = 0;
    for(int i = 0; i < cnt; i++)
        if(k >> i & 1)
            res ^= p1[i];
    return res;
}
```

询问存在性：把 x 尝试插进线性基，如果可以插入，说明不能异或得到，否则说明可以异或得到

```
bool query(int x) {
    for(int i = B - 1; i >= 0; i--)
        if(x >> i & 1) {
            if(!p[i]) return false;
            x ^= p[i];
        }
    return true;
}
```

找最小

题意：你有两个长度为 n 的序列 a , b ，你可以进行若干次选择一个 i $(1 \le i \le n)$ ，然后交换 a_i , b_i ，希望使得 $max\{f(a), f(b)\}$ 最小，其中对于一个序列 A ，$f(A) = \oplus_{i=1}^{n} A_i$

数据范围：$1 \le T \le 10^5$ ，$1 \le n \le 10^6$ ，$0 \le a_i, b_i < 2^{31}$ ，$\sum n \le 10^6$

题解：先求出两个序列分别的异或和记为 A , B ，则修改位置 i 相当于让两者同时异或 $a_i \oplus b_i$ ，则可以将所有 $a_i \oplus b_i$ 构造线性基。接下来从 A , B 的高位往低位考虑，对于一位：

- 如果都是 1 ，且线性基这一位有值，则同时异或线性基这一位，都变成 0 肯定更优
- 如果都是 0 ，则不管
- 如果一个 1 一个 0 ，后面肯定让这一位是 1 的尽可能小，0 的不用管，则用线性基贪心即可。如果这一位线性基有值，那么需要枚举是否异或这个值，两种情况都往下贪一次即可

时间复杂度：$O(n \log w)$

```
const int B = 32;
int p[B];

void insert(int x) {
    for(int i = B - 1; i >= 0; i--)
        if(x >> i & 1) {
            if(!p[i]) {
                p[i] = x;
```

```
                break;
            }
            x ^= p[i];
        }
}

int work(int sa, int sb, int pos) {
    for(int i = pos; i >= 0; i--) {
        assert(sa > sb);
        int a = sa >> i & 1, b = sb >> i & 1;
        if(a && p[i]) sa ^= p[i], sb ^= p[i];
    }
    return max(sa, sb);
}

void solve() {
    for(int i = 0; i < B; i++)
        p[i] = 0;

    int n;
    cin >> n;
    vector<int> a(n), b(n);
    for(int i = 0; i < n; i++)
        cin >> a[i];
    for(int i = 0; i < n; i++)
        cin >> b[i];
    int sa = 0, sb = 0;
    for(int i = 0; i < n; i++) {
        sa ^= a[i], sb ^= b[i];
        insert(a[i] ^ b[i]);
    }

    int res = max(sa, sb);
    for(int i = B - 1; i >= 0; i--) {
        int a = sa >> i & 1, b = sb >> i & 1;
        if(a && b) {
            if(p[i]) sa ^= p[i], sb ^= p[i];
        } else if(a) {
            res = min(res, work(sa, sb, i - 1));
            if(p[i]) res = min(res, work(sb ^ p[i], sa ^ p[i], i - 1));
            break;
        } else if(b) {
            res = min(res, work(sb, sa, i - 1));
            if(p[i]) res = min(res, work(sa ^ p[i], sb ^ p[i], i - 1));
            break;
        }
    }
```

```
    res = min(res, max(sa, sb));
    cout << res << '\n';
}
```

鱼的记忆只有 3 秒

题意：你被给定 n 个整数 $a_1, a_2, \ldots, a_n$，需要执行以下操作恰好 $n-1$ 次：选择序列中的两个整数 x 和 y，移除它们，并添加一个值为 $x \oplus y$ 的整数。仅仅这样操作太无聊了，因此你可以在任意时刻选择序列中的一个整数，并将它加一。你必须执行这种加一操作恰好一次，最终序列将剩下一个整数，而你则需要最大化这个剩余的整数。请输出这个剩余的整数的最大值

数据范围：$1 \le n \le 10^6$，$0 \le a_i < 2^{60}$

题解：注意到，对于二进制表示为 $x = (????0\underbrace{11\cdots11}_{k})_2$ 的整数（含义：末尾有连续 k 个 1，且这 k 个 1 的前一位为 0，高位无限制），满足等式：$x + 1 = x \oplus (2^{k+1} - 1)$。因此我们可以枚举 k 的取值，判断是否能得到形如 $x = (????0\underbrace{11\cdots11}_{k})_2$ 的数值，这一验证过程可以通过线性基完成

时间复杂度：$O(nw^2)$，该算法还可用进一步优化至 $O((n+w)\cdot w)$，但本题不强制要求实现优化版本

```
const int B = 64;
int p[B];

void insert(int x) {
    for(int i = 0; i < B; i++)
        if(x >> i & 1) {
            if(!p[i]) {
                p[i] = x;
                break;
            }
            x ^= p[i];
        }
}

bool query(int x, int pos) {
    for(int i = 0; i < pos; i++)
        if(x >> i & 1) {
            if(!p[i]) return false;
            x ^= p[i];
        }
    if((x >> pos & 1) && p[pos]) x ^= p[pos];
    return (x >> pos & 1) == 0;
}
```

```

void solve() {
    int n;
    cin >> n;
    int sum = 0;
    for(int i = 0, x; i < n; i++) {
        cin >> x;
        sum ^= x;
        insert(x);
    }

    int res = 0;
    for(int i = 0, t = 0; i + 1 < B; i++, t = (t << 1) | 1)
        if(query(t, i))
            res = max(res, sum ^ ((t << 1) | 1));
    cout << res << '\n';
}
```

Gemcrate

题意：有 n 颗宝石，第 i 颗宝石上刻有正整数 a_i，将这些宝石分成若干个非空组，每颗宝石恰好属于一个组。假设第 i 组包含编号为 $k_{i,1}, k_{i,2}, \ldots, k_{i,p}$ 的宝石，那个第 i 组的亮度定义为：$a_{k_{i,1}} \oplus a_{k_{i,2}} \oplus \cdots \oplus a_{k_{i,p}}$。对于一个分成 m 个组的分组方案，该方案的价值定义为 $B_1 \& B_2 \& \ldots \& B_m$，求所有分组方案中能得到的最大价值

数据范围：$1 \le T \le 10^4$，$1 \le n \le 5 \times 10^5$，$1 \le a_i < 2^{60}$，$\sum n \le 5 \times 10^5$

题解：首先证明存在组数不超过 2 的最优分组方案：若最优分组方案有至少 3 组，且答案的某一位为 1，则所有组假值的对应位均为 1，此时选择其中任意三组，将前两组中的所有数合并到最后一组，则易得答案的对应位依然为 1，不会使答案变劣

- 最高位为 1 的数字数量为奇数：将所有数分为一组即可
- 最高位为 1 的数字数量为偶数：需要将所有数分为两组。此时原问题等价于最大异或子集，使用贪心线性基即可解决

```
const int B = 64;
int p[B];

void insert(int x) {
    for(int i = B - 1; i >= 0; i--)
        if(x >> i & 1) {
            if(!p[i]) {
                p[i] = x;
                break;
```

```
            }
            x ^= p[i];
        }
}

int max_xor() {
    int res = 0;
    for(int i = B - 1; i >= 0; i--)
        if((res ^ p[i]) > res)
            res ^= p[i];
    return res;
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    int sum = 0;
    for(int i = 0; i < n; i++) {
        cin >> a[i];
        sum ^= a[i];
    }

    // 问题转换为 max(v & (sum ^ v))
    // 对某一位 pos, 只有当 sum_pos = 0, 才可以有 v_pos = 1
    for(int i = 0; i < B; i++)
        p[i] = 0;
    for(int i = 0; i < n; i++)
        insert(a[i] & (~sum));
    int res = max(max_xor(), sum);
    cout << res << '\n';
}
```

月球异或

题意：定义月球异或运算为三进制下各位在模 3 意义下相加，给定一个集合，多次询问一个数字能否被集合中若干数字月球异或得到（集合内同个数字可以选多次）

数据范围：$1 \le T \le 100000$，$1 \le N, Q \le 10^6$，$0 \le v_i, s_i \le 10^{18}$，$1 \le N_{tot}, Q_{tot} \le 2 \times 10^6$

题解：题意可以转化为在一个集合中选择若干元素（一个元素可以被选择多次），加上询问的数字 s 后所有数字在三进制下，各位的总和模 3 意义下为 0。考虑构建一个特殊的线性基，我们就可以从高到低控制三进制下每一位的值，需要注意这一位是 1 还是 2，注意到所有数字都可以通过自选两次把当前为的 2 统一化为 1，可能会易于代码实现

时间复杂度：$O(n \log V)$

```
const int B = 38;
int p[B][3], pow3[B];

int add(int x, int y) {
    int res = 0;
    for(int i = 0; i < B; i++) {
        int a = x % 3, b = y % 3;
        x /= 3, y /= 3;
        int v = (a + b) % 3;
        res += v * pow3[i];
    }
    return res;
}

int del(int x, int y) {
    int res = 0;
    for(int i = 0; i < B; i++) {
        int a = x % 3, b = y % 3;
        x /= 3, y /= 3;
        int v = (a - b + 3) % 3;
        res += v * pow3[i];
    }
    return res;
}

void insert(int x) {
    for(int i = B - 1; i >= 0; i--) {
        int c = (x / pow3[i]) % 3;
        if(c) {
            if(!p[i][c]) {
                p[i][c] = x;
                break;
            }
            x = del(x, p[i][c]);
        }
    }
}

bool query(int x) {
    for(int i = B - 1; i >= 0; i--) {
        int c = (x / pow3[i]) % 3;
        if(c) {
            if(!p[i][c]) return false;
            x = del(x, p[i][c]);
        }
```

```
    }
    return true;
}

void solve() {
    for(int i = 0; i < B; i++)
        p[i][1] = p[i][2] = 0;

    int n, m;
    cin >> n >> m;
    for(int i = 0, x; i < n; i++) {
        cin >> x;
        insert(x);
        x = add(x, x);
        insert(x);
    }
    while(m --) {
        int x;
        cin >> x;
        x = del(0, x);
        if(query(x)) cout << "Yes\n";
        else cout << "No\n";
    }
}
```